

TLS - Part II

CS 161 Spring 2026 - Lecture 30

Reminder: TLS has two sub-protocols





Handshake = establish shared secret key

Key *K*

Key *K*

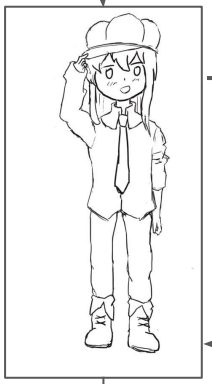
Record Layer = bulk data transfer

Last time: Simplified handshake

Parameters:
– Diffie-Hellman params *g* and *p*

PK_{CA}

– Pick random nonce *r*
– Pick Diffie-Hellman secret *a*
– Set $A \leftarrow g^a \bmod p$



(r, A)

SK_{berk} $Cert_{berk}$



– Pick Diffie-Hellman secret *b*
– Set $B \leftarrow g^b \bmod p$
– $\sigma \leftarrow \text{Sign}(SK_{Berk}, (r, A, B, \text{“berk.edu”}))$

$(B, \ \sigma, Cert_{berk})$

– Verify $Cert_{Berk}$ against PK_{CA}
– Extract identity (“berk.edu”) from $Cert_{Berk}$
– Verify σ against $Cert_{Berk}$

$K \leftarrow \text{Hash}(A, B, B^a \bmod p, \text{“berk.edu”})$

$K \leftarrow \text{Hash}(A, B, A^b \bmod p, \text{“berk.edu”})$

Reminder: TLS has two sub-protocols





Handshake = establish shared secret key

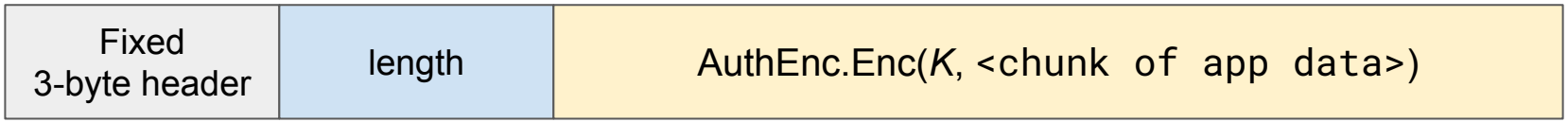
Key *K*

Key *K*

Record Layer = bulk data transfer

TLS 1.3 record layer

- Each party sends data in TLS “records” with <= 16 KB of data each
  - **Question:** Why only 16 KB?
- Both parties derive symmetric encryption key from handshake key
- Each side encrypts data using an authenticated-encryption scheme
  - Plus one extra tweak to save some bandwidth



How an application interacts with TLS


```
import socket
import ssl
context = ssl.create_default_context()

# Open TCP connection
sock = socket.create_connection((“berkeley.edu”, 443))

# TLS: Run handshake
tls_sock = context.wrap_socket(sock, server_hostname=”berkeley.edu”)

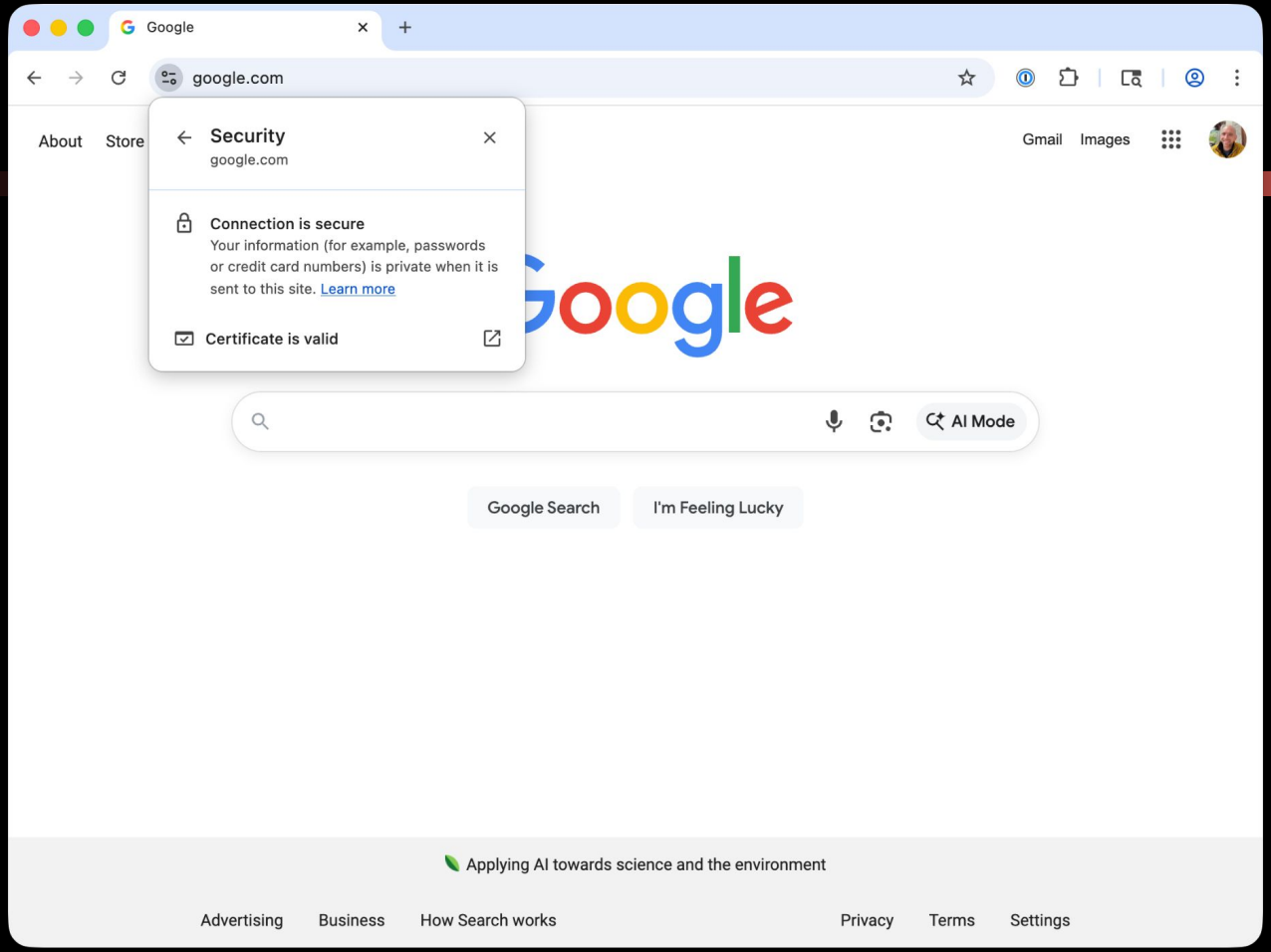
# TLS: Send record-layer traffic
tls_sock.send_all(b”GET / HTTP/1.1 ...”)

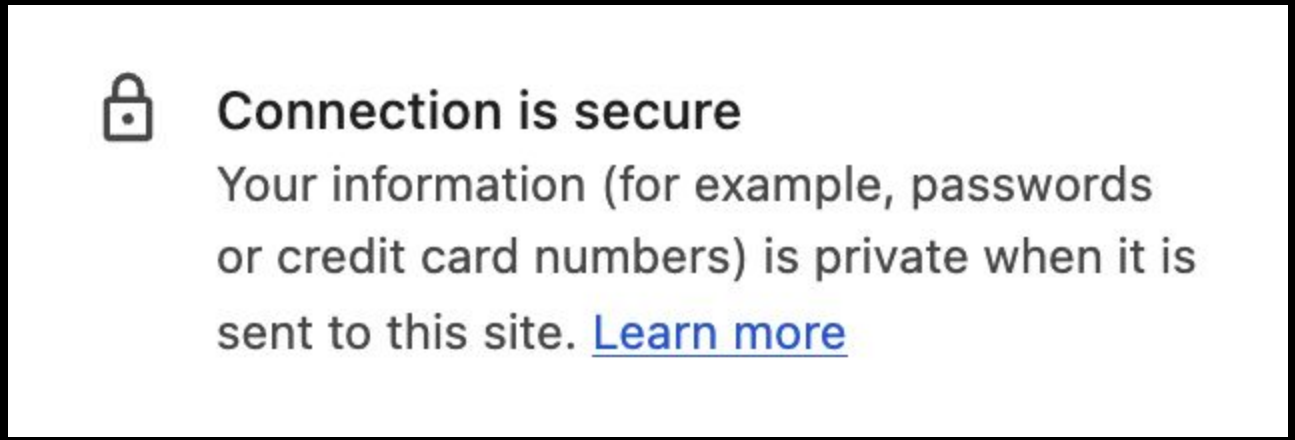
# TLS: Recv record-layer traffic
data = tls_sock.read()
```

TLS solved all of the world's network-security problems and the computers lived happily ever after.

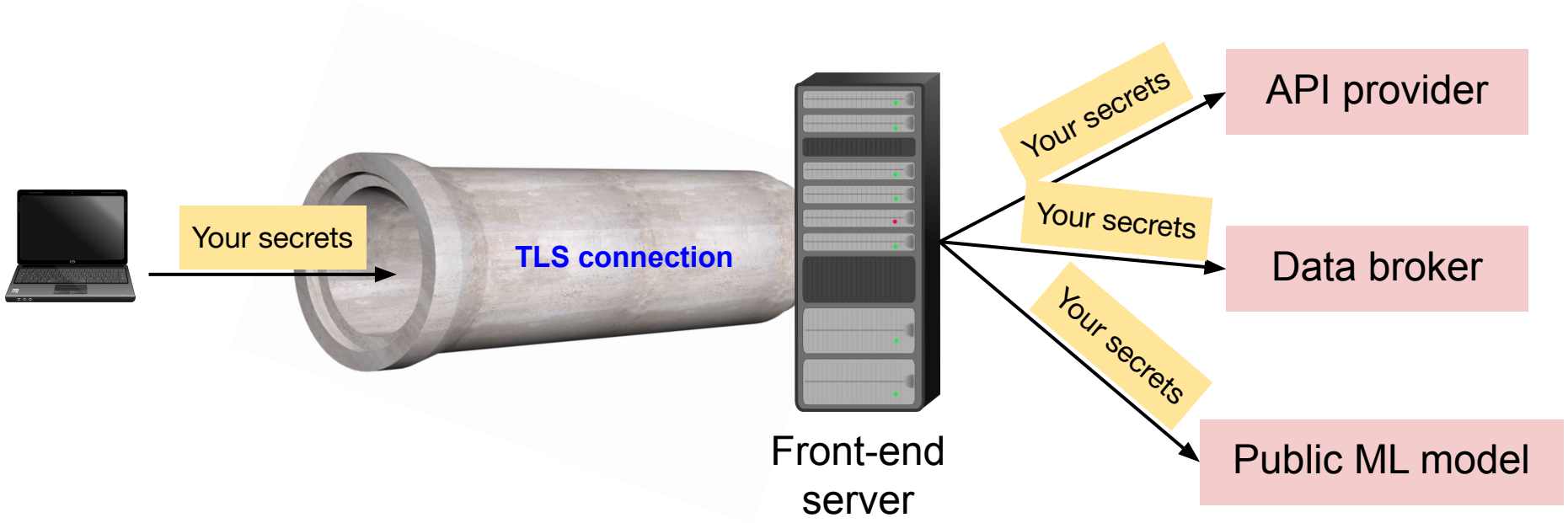
~~The end.~~ **Not quite . . .**

Limitations of TLS

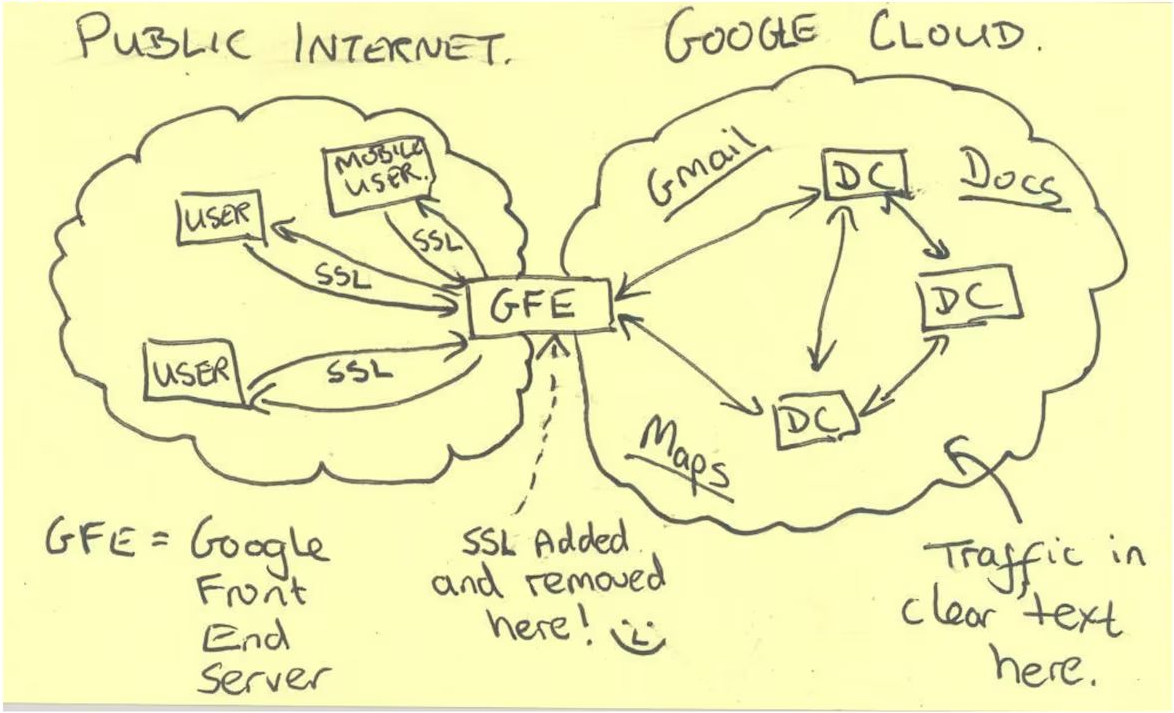

Google
google.com
About
Store
Gmail
Images
Security
google.com
Connection is secure
Your information (for example, passwords or credit card numbers) is private when it is sent to this site. Learn more
Certificate is valid
Google
AI Mode
Google Search
I'm Feeling Lucky
Applying AI towards science and the environment
Advertising
Business
How Search works
Privacy
Terms
Settings


Connection is secure
Your information (for example, passwords or credit card numbers) is private when it is sent to this site. Learn more

TLS does not prevent the server from **leaking your secrets**

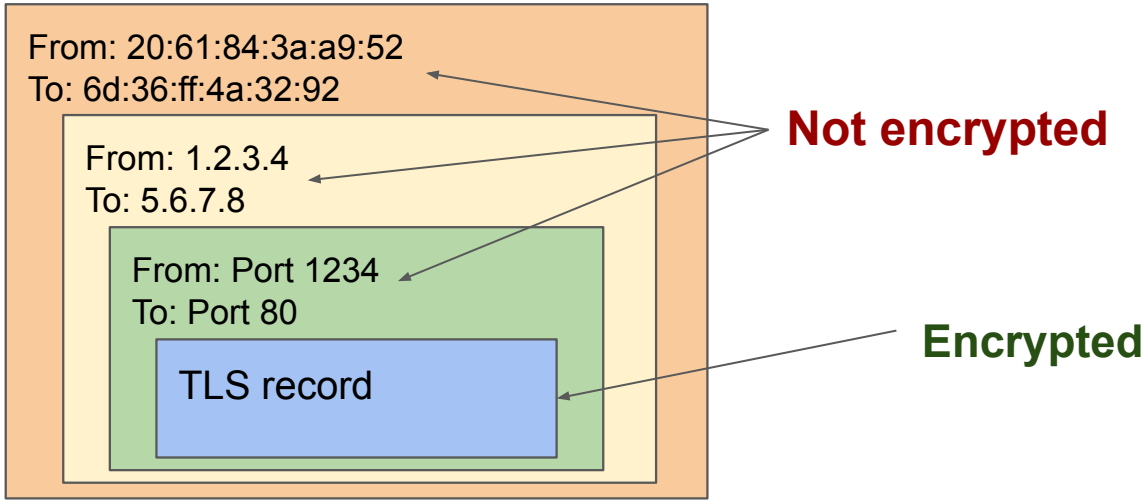


Current Efforts - Google



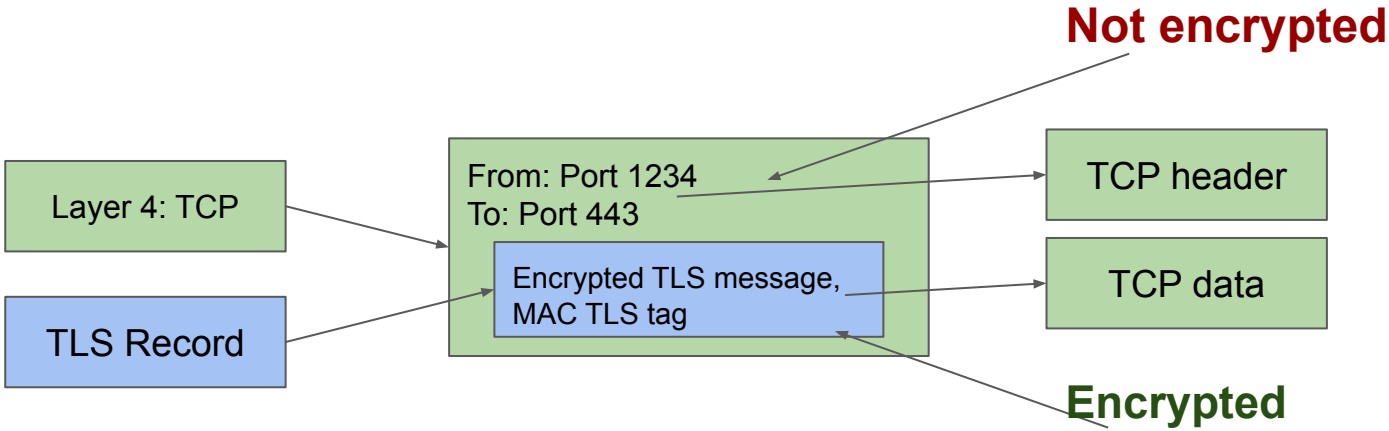
TLS does not hide **who you are talking to**

- Revealed via headers: IP address, MAC address, port number
- The client typically includes the server name **unencrypted** in the TLS ClientHello
  (Next week: How to hide who you are talking to)



TLS does not protect **availability**

- An attacker can close a TLS connection
  - By dropping packets or by spoofing TCP reset
- Result: A censor can selectively block TLS connections
  - Can selectively block based on domain name
  - Some governments do this to block “objectionable” websites



TLS does not guarantee **that you will get the "whole message"**

- **Guarantee of TLS:** The application only sees authenticated data

```
# TLS: Recv record-layer traffic
data = tls_sock.read()
```

- Does NOT guarantee that the receiver will get all data that the sender sent!

```
# On sender
tls_sock.send_all(movie_file_that_is_4gb_long)

# On recipient
data = tls_sock.read()
```

Data might be only 3KB long if TCP connection closes unexpectedly (e.g., network failure)

TLS does not guarantee **that you will get the “whole message”**

Does it matter if recipient doesn’t get the whole message? **MAYBE**

Example: Installing Claude code

```
curl https://claude.ai/install.sh | bash
```

- `curl` fetches an install script over TLS
- If connection drops, `bash` will execute whatever it received so far
- Guarantee of TLS: `bash` will execute **a prefix of** `install.sh`
  - Is that okay? **MAYBE.**

```
rm -rf “/tmp/stuff.bin”;      rm -rf /
```

TLS does not protect the **length of the encrypted data**

CRIME Attack

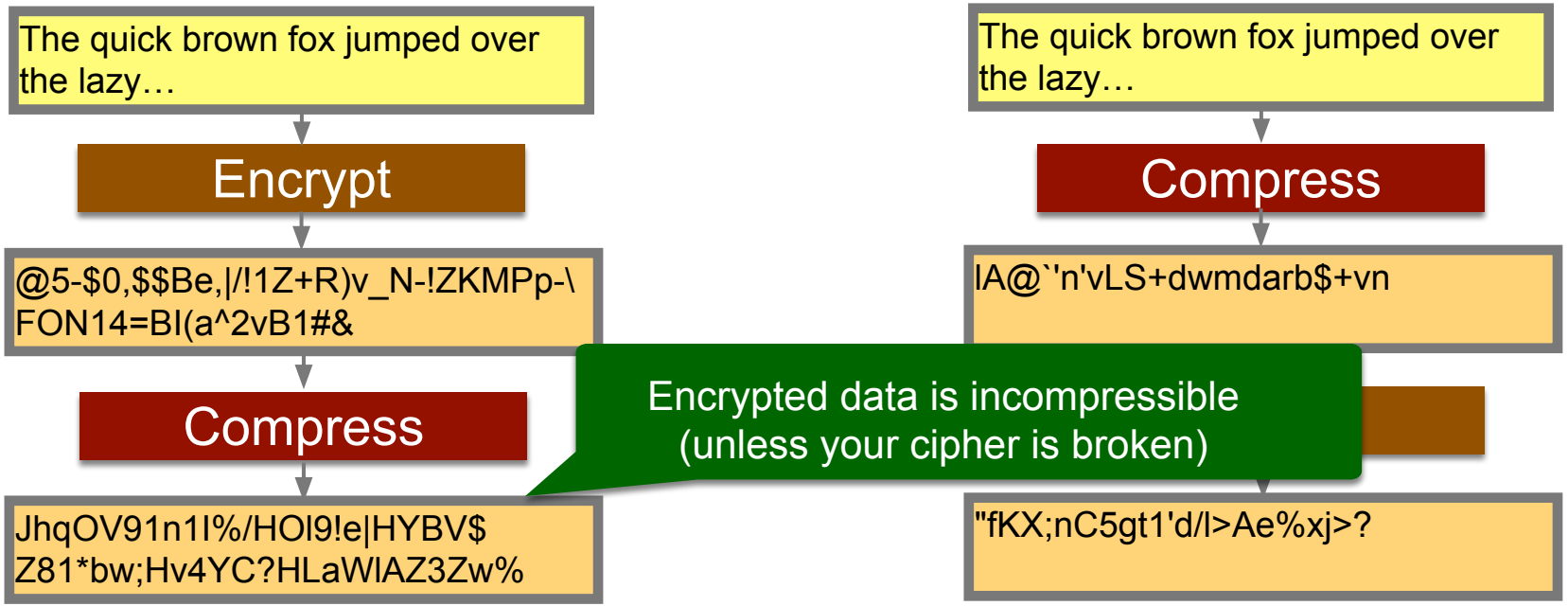
Compression and encryption

- We often send compressible data (e.g., ASCII text)
- Compressed data -> better apparent network throughput
- TLS 1.2 and earlier supported TLS record compression



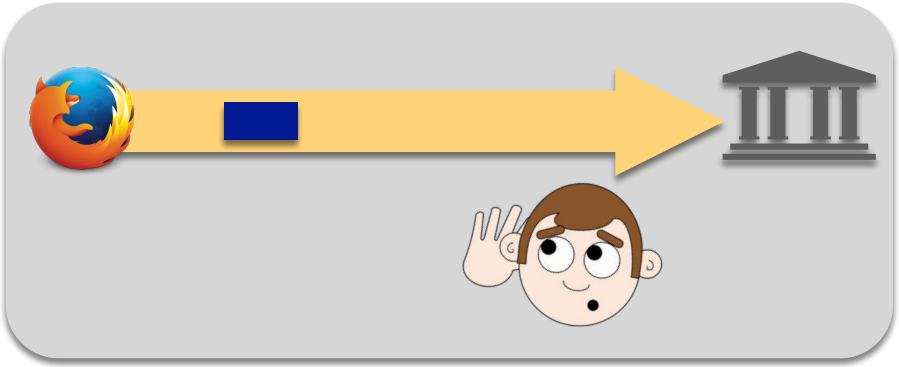
Compression and TLS

Encrypt then compress? Or, compress then encrypt?

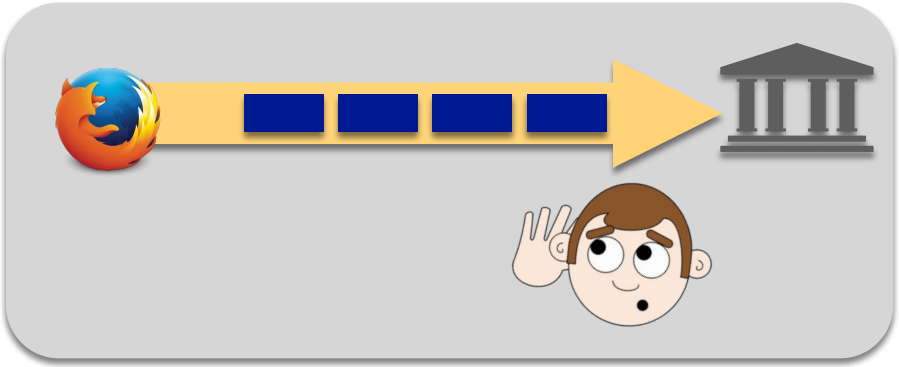


Compress-then-encrypt

PROBLEM: Ciphertext length leaks information to attacker.



Message is compressible.



Message not compressible.

Compress-then-encrypt

PROBLEM: Ciphertext length leaks information to attacker.

- Simple example: attacker can tell if you're sending the string
  000000…00000 versus random string
- More interesting: attacker can tell if you're streaming
  "Mad Men" versus "Sopranos"
- Does this really matter?

Attacking compress-then-encrypt

- When you log into bank.com, your bank stores a “cookie” in browser
  - Cookie could be a 128-bit random string
  - Anyone with your bank cookie can access your account

- If you visit evil.com, an attacker can trick your browser into sending many encryptions of: “attacker-chosen-string || `Cookie:secret-string`”

```
<html><title>Evil.com</title>
<img src="https://bank.com/?attacker-chosen-string1">
<img src="https://bank.com/?attacker-chosen-string2">
<img src="https://bank.com/?attacker-chosen-string3">
...
```

Cookie sent with these requests

Attacking compress-then-encrypt

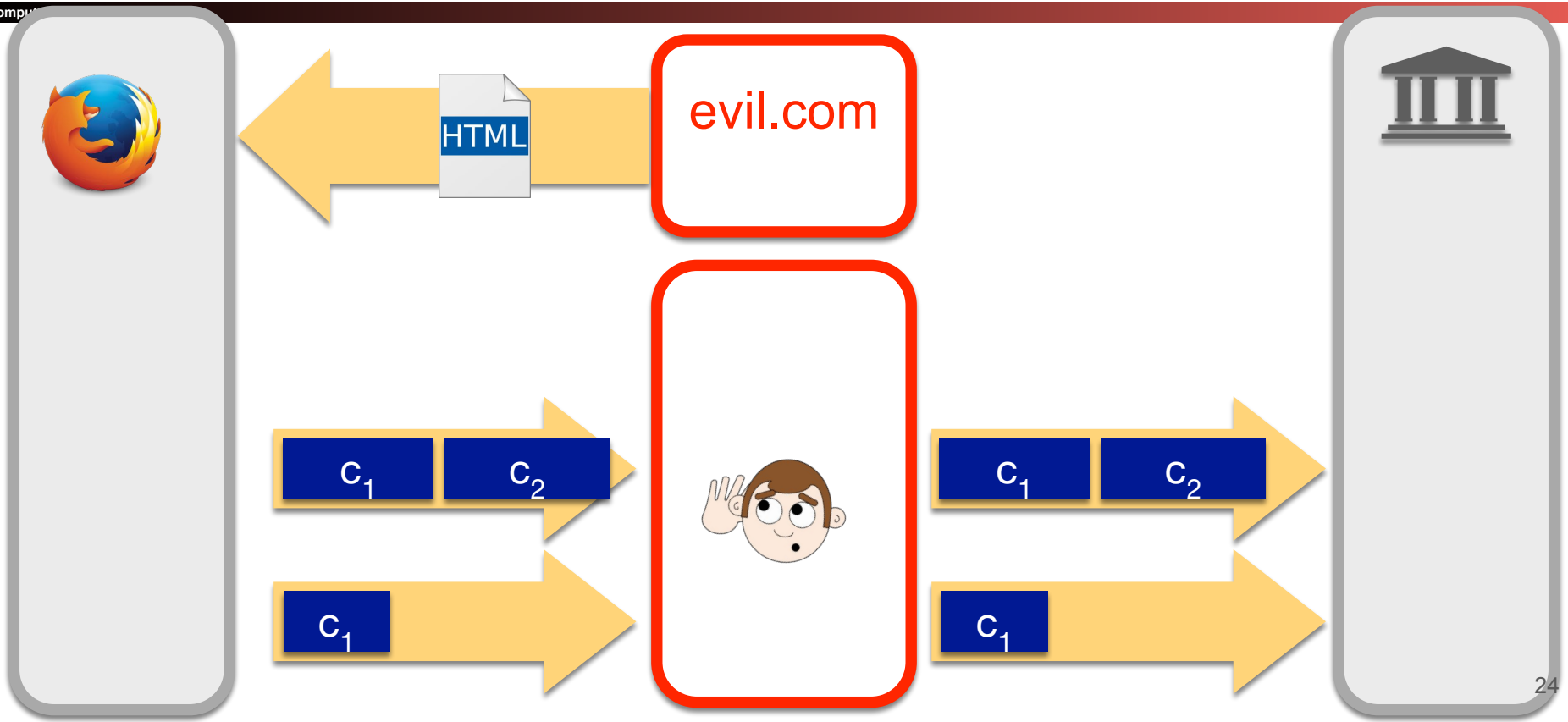
Attack idea (CRIME and BREACH TLS attacks)

[Rizzo&Duong'12] [Prado&others'13]

1. For many choices of s, trick user into sending encryptions of
   `<s || Cookie:secret-bank-token>`
2. Observe how the length of the encrypted string varies.
3. Recover the password.

https://www.ietf.org/proceedings/85/slides/slides-85-saag-1.pdf

Attack idea



Attack idea

Say that the cookie is `HX8f$Q`. Then

`Cookie:ACookie:HX8f$Q`	compresses to	24 bytes
`Cookie:BCookie:HX8f$Q`		24 bytes
`Cookie:CCookie:HX8f$Q`		24 bytes
…		
`Cookie:HCookie:HX8f$Q`		23 bytes
`Cookie:HACookie:HX8f$Q`		24 bytes
`Cookie:HBCookie:HX8f$Q`		24 bytes
`Cookie:HCCookie:H`		24 bytes
…		
`Cookie:HXCookie:HX8f$Q`		23 bytes

Attacker knows that

Attacker knows that cookie starts with “HX”

Compress-then-encrypt

- For cookie of length L:
  - Cost of this attack: 256L work
  - Brute-force guessing: 256^L work

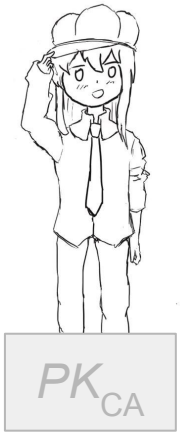
Exponential speed-up!

- These attacks are practical-ish!

Mitigation?

Turn off compression. :(Most TLS servers/clients don't compress.
  - Can still use application-layer compression (e.g., in HTTP2)

TLS does not protect against **certificate authority** compromise



Compromised Certificate authority (SK_{CA}, PK_{CA})

SK_{CA}

ATTACKER (SK_{evil}, PK_{evil})

TLS does not protect against **certificate authority** compromise

Certificate authority

(SK_{CA}, PK_{CA})



PK_{CA}

Certificate

(σ, "berkeley.edu owns PK_{evil}")

ATTACKER

(SK_{evil}, PK_{evil})

TLS does not protect against **certificate authority** compromise

Alice thinks she is talking to berkeley.edu, but she is talking to attacker



PK_{CA}

Verify(PK_{CA}, "berkeley.edu owns PK_{evil}", σ) = 1 **?**

ATTACKER

(SK_b, PK_b)

COMPUTERWORLD

Link

Solo Iranian hacker takes credit for Comodo certificate attack

Gregg Keizer *March 27, 2011*

Security researchers split on whether 'ComodoHacker' is the real deal

A solo Iranian hacker on Saturday claimed responsibility for stealing multiple SSL certificates belonging to some of the Web's biggest sites, including Google, Microsoft, Skype and Yahoo.

Early reaction from security experts was mixed, with some believing the hacker's claim, while others were dubious.

c|net

Link

Fraudulent Google certificate points to Internet attack

Elinor Mills *August 29, 2011*

Is Iran behind a fraudulent Google.com digital certificate? The situation is similar to one that happened in March in which spoofed certificates were traced back to Iran.

A Dutch company appears to have issued a digital certificate for Google.com to someone other than Google, who may be using it to try to re-direct traffic of users based in Iran.

Yesterday, someone reported on a Google support site that when attempting to log in to Gmail the browser issued a warning for the digital certificate used as proof that the site is legitimate, according to this thread on a Google support forum site.

threatpost

Link

Final Report on DigiNotar Hack Shows Total Compromise of CA Servers

Dennis Fisher *October 31, 2012*

The attacker who penetrated the Dutch CA DigiNotar last year had complete control of all eight of the company's certificate-issuing servers during the operation and he may also have issued some rogue certificates that have not yet been identified.

techdirt

Link

Evidence Suggests DigiNotar, Who Issued Fraudulent Google Certificate, Was Hacked *Years* Ago

Mike Masnick *August 30, 2011*

The big news in the security world, obviously, is the fact that a fraudulent Google certificate made its way out into the wild, apparently targeting internet users in Iran. The Dutch company DigiNotar has put out a statement saying that it discovered a breach back on July 19th during a security audit, and that fraudulent certificates were generated for "several dozen" websites. The only one known to have gotten out into the wild is the Google one.

Protecting against CA compromise

- **Certificate pinning**: The browser restricts which CAs it will use for certain sites
  - Example: Only Google's favorite CA can sign certificates for Google websites
  - Now creating a fake certificate for a specific site requires attacking a particular CA
  - Problem: Does not scale well

- **Certificate transparency**: Public logging of all issued certs
  - Specifics are out of scope
  - High-level idea: CAs must publish all certs they issue in a public log
  - Browser only accepts certificates from CAs implementing transparency
  - If CA issues a bad certificate, everyone can see it in the public logs

Transport-layer security

TLS = End-to-end encryption for any application that runs over TCP

Lots of limitations

- Only protects traffic to the host's front-end server
- Doesn't protect communications metadata (who is talking to whom)
- Leaks number of bytes sent
- Relies on brittle CA infrastructure

→ Still, TLS is the best tool we have for this use case

Next time: Encrypted messaging!